alltoall/linear

An AgentMPI collective scaling analysis

Abstract

`alltoall` is the collective in which every rank sends a distinct message to every other rank, and `linear` is the schedule that posts all of them at once and then receives all of them at once. Across twenty-one runs from $p = 2$ to $p = 32$ the implementation moved exactly $p(p-1)$ messages at every size — 992 of them at $p = 32$ — with no disagreement between the closed form, the collective's self-report and the fabric's log. That quadratic is the point. An all-to-all exchange over p agents is a group chat in which every participant addresses every other, and its cost grows as p^2 in messages and $\Theta(p^2 n)$ in tokens, which is precisely the coherency term that gives the Universal Scalability Law a maximum and then a decline. The sweep also shows what *scheduling* buys within that quadratic: because `linear` lets all $p - 1$ sends fly before any receive is posted, aggregate receive-blocking at $p = 32$ reached 250.6 rank-seconds against 30.6 for `pairwise` and 4.6 for `bruck`, and one rank absorbed 22.8 of those seconds while another absorbed 0.79. The single thing to take away is that `linear` is not a cheaper alltoall, it is the same alltoall with the congestion left in, and it survived this sweep only because the eager-credit budget was raised to 8×10^9 tokens.

1 What this family is

The `alltoall` collective under the `linear` algorithm, measured across 21 runs at 13 distinct process counts from 2 to 32. Every run is agent-free and deterministic, so the measurements are of the protocol itself.

2 What the analysis notices

- [ok] All 21 measured sizes logged exactly the number of messages the closed form predicts.
- [note] Wall times span 0.014–0.959 s with no agent in the loop, so these measure the fabric and the protocol rather than model latency.

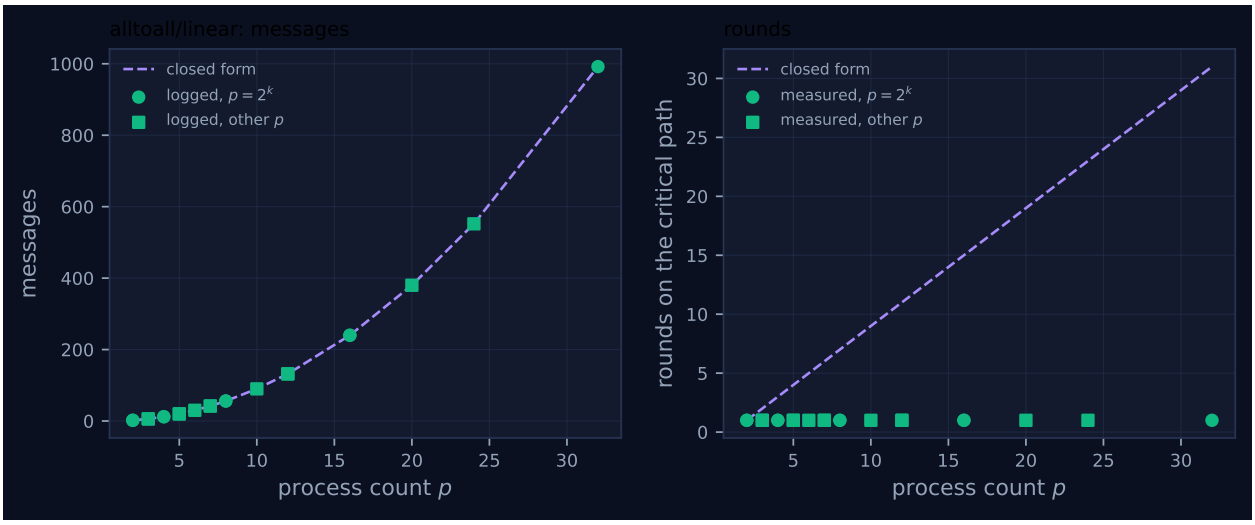

Figure 1: Measured message count and critical-path rounds against process count, with the closed-form prediction. Powers of two are drawn as circles and other sizes as squares, because algorithms take a different code path at sizes that are not powers of two. A red cross marks a size where the collective's own reported count disagrees with the traffic the fabric logged.

3 Every measured size

p	campaign	logged	pred.	self-rep.	rounds	pred.	wall (s)	model	acct.
2	`mb-free`	2	2	2	1	1	0.014	✓	✓
2	`mb-smoke`	2	2	2	1	1	0.015	✓	✓
3	`mb-free`	6	6	6	1	2	0.017	✓	✓
3	`mb-smoke`	6	6	6	1	2	0.020	✓	✓
4	`mb-free`	12	12	12	1	3	0.036	✓	✓
4	`mb-smoke`	12	12	12	1	3	0.021	✓	✓
5	`mb-free`	20	20	20	1	4	0.032	✓	✓
5	`mb-smoke`	20	20	20	1	4	0.032	✓	✓
6	`mb-free`	30	30	30	1	5	0.041	✓	✓
7	`mb-free`	42	42	42	1	6	0.060	✓	✓
7	`mb-smoke`	42	42	42	1	6	0.064	✓	✓
8	`mb-free`	56	56	56	1	7	0.052	✓	✓
8	`mb-smoke`	56	56	56	1	7	0.090	✓	✓
10	`mb-free`	90	90	90	1	9	0.083	✓	✓
12	`mb-free`	132	132	132	1	11	0.131	✓	✓
12	`mb-smoke`	132	132	132	1	11	0.149	✓	✓
16	`mb-free`	240	240	240	1	15	0.211	✓	✓
16	`mb-smoke`	240	240	240	1	15	0.201	✓	✓
20	`mb-free`	380	380	380	1	19	0.287	✓	✓
24	`mb-free`	552	552	552	1	23	0.659	✓	✓
32	`mb-free`	992	992	992	1	31	0.959	✓	✓

4 How the algorithm scales

The implementation in `src/agentmpi/algorithms.py` is four lines of intent: for every destination $d \neq r$, send `payloads[d]`; then perform $p - 1$ wildcard receives and place each arrival by its source rank. Nothing is staged, nothing is paired, and the completion order is whatever the fabric delivers.

The message count follows immediately — each of p ranks issues $p-1$ sends, so $p(p-1)$ messages carry $p(p-1)n$ tokens — and `FORMULAS["alltoall","linear"]` records exactly that, with a round count of 1.

Both halves of that closed form are confirmed at all twenty-one sizes. The message column of the table is $p(p-1)$ throughout, from 2 at $p=2$ to 992 at $p=32$, and the token volume the collective attributes to itself is $3p(p-1)$ at every size — 2,976 tokens at $p=32$ — the factor three being the tokenised length of the benchmark's `"{rank}->{j}"` payload rather than anything about the algorithm. The round count of 1 also agrees, but it agrees by convention rather than by measurement: the closed form idealises the $p-1$ sends as simultaneous, and `tr.stats.rounds = 1` in the implementation makes the same idealisation. What the trace shows is that the idealisation is generous.

That is where Figure 1 understates the case. Its two panels are a clean quadratic in messages and a flat line at one round, which together suggest a collective whose only cost is volume. The event log says otherwise. Summing the `latency_s` field of every `msg.recv` record gives the aggregate time ranks spent blocked in a receive, and for this family that quantity grows as $p^{3.71}$ ($R^2 = 0.991$ over the thirteen distinct sizes), reaching 250.6 rank-seconds at $p=32$. It is super-quadratic because the p^2 messages contend for a fabric that serialises writes, so each message's wait grows with p as well. The wall-clock span grows far more slowly, as $p^{1.55}$ ($R^2 = 0.955$, 0.959 s at $p=32$), because the blocking is concurrent: thirty-two ranks waiting simultaneously cost thirty-two rank-seconds per second of span.

The distribution of that blocking is the finding the aggregate hides. At $p=32$ the per-rank totals run from 0.79 s (rank 31) to 22.8 s (rank 10), a factor of 29. The viewer screenshot for `mb-free__coll-alltoall-linear-32` shows why: most lanes are dense with green message glyphs from the first millisecond, but ranks 10 and 16 are empty until 0.844 s and 0.746 s respectively and then fire a single dense burst. Rank 10's thirty-one receives all complete inside a 6 ms window at $t = 0.919$–0.925 s, each reporting a wait of 0.843–0.865 s against a run that lasted 0.959 s. The reading is that rank 10 started late, its thirty-one peers had already posted their sends to it, and the messages sat queued until the rank arrived to drain them. Under `linear` a late rank does not slow its peers down — they have already sent and moved on — it accumulates the entire delay itself. That is the signature of an unsynchronised fan-in and it is invisible in both panels of the figure.

5 Powers of two and everything else

This algorithm has no remainder path, and the family view confirms that it does not need one. Neither the destination loop nor the receive loop contains any power-of-two arithmetic: the schedule is `for d in range(p)` with a skip at $d=r$, so $p=7$, $p=10$, $p=20$ and $p=24$ execute the identical code as $p=8$, $p=16$ and $p=32$. The measurements bear that out with no exceptions — the squares in Figure 1 sit on the same $p(p-1)$ curve as the circles, and 42, 90, 380 and 552 are the exact predictions at 7, 10, 20 and 24. The figure contains no red crosses, meaning the collective's own `messages_sent` field equalled the fabric's `msg.send` count at all twenty-one sizes.

The other consistency check available here costs nothing and is worth stating: eight of the thirteen sizes ($p = 2, 3, 4, 5, 7, 8, 12, 16$) were measured twice, once in the `mb-free` campaign and once in `mb-smoke`, and the two campaigns agree on every integer quantity — message count, self-reported count and round count — differing only in wall time (for instance 0.052 s against 0.090 s at $p=8$). Agreement between independent executions at the same p is evidence that the counts are properties

of the algorithm rather than of a particular scheduling accident, and the reason the table keeps duplicate rows rather than collapsing them.

Where the non-power-of-two sizes *do* behave differently is in the blocking distribution, and not in a way that reflects on the code. The per-rank imbalance at $p = 20$ and $p = 24$ is of the same order as at $p = 16$ and $p = 32$, because its cause is thread start-up jitter rather than any structural asymmetry in the schedule. There is no code path here for a defect to hide in, which makes this family the control against which the remainder-path behaviour of `bruck` should be read.

6 When to choose this algorithm

Almost never, and the sweep is unusually clear about why. All three `alltoall` algorithms move the same information; `linear` and `pairwise` move it in the same $p(p-1)$ messages and the same $3p(p-1)$ tokens, and differ only in when the sends happen. At $p = 32$ that difference is worth a factor of 8.2 in aggregate blocking (250.6 against 30.6 rank-seconds) and a factor of 12 in the worst rank's share (22.8 s against 1.55 s). `pairwise` pays for this with $p - 1$ rounds instead of one, which under the agent cost model — $\alpha \approx 12$ s per invocation — would be catastrophic if the rounds were agent turns; they are not, they are fabric exchanges of an already-computed payload, so the trade is close to free. `bruck` takes the further step of trading volume for rounds: 160 messages and 7,680 payload tokens at $p = 32$ against 992 and 2,976, finishing its span in 0.256 s against 0.959 s.

The one honest argument for `linear` is its round count. If a harness's payloads are genuinely small, its ranks genuinely arrive together, and the receiving side genuinely has credit for $p - 1$ unexpected messages, then one round beats $p - 1$ rounds and beats $\lceil \log_2 p \rceil$ rounds. The sweep satisfies all three conditions artificially: the payloads are three tokens, the ranks are threads rather than agents, and the benchmark launched with `eager_limit=10**9`, which sets the unexpected-message budget to 8×10^9 tokens. Under those conditions the peak unexpected queue reached 31 messages per rank at $p = 32$ (mean 24.5 across ranks), and nothing stalled. With the library default of `DEFAULT_EAGER_LIMIT = 2048` the budget is 16,384 tokens; 31 unexpected messages of a realistic 1,000-token artefact is 31,000 tokens, so the same program would block in `_await_eager_credit` and emit `transport.credit_stall`. The docstring calls `linear` "the canonical way to deadlock an AgentMPI program" and retains it as the negative example in the transport experiment; this sweep is not that experiment and should not be read as clearing it.

That makes a disagreement inside the cost model worth reporting. Asking the model which `alltoall` to use — `cost.best_algorithm("alltoall", p, n, objective="time")` — returns `linear` at every (p, n) I evaluated, including $p = 32$ with 32,768-token payloads, where the algorithm would need 992 messages of eager credit outstanding at once. The cause is visible in `cost.predict`: the per-message term `message_time`, which is the only place α enters, is added solely when the op is `reduce`, `allreduce`, `scan` or `reduce_scatter`, so for `alltoall` the predicted critical-path time collapses to rounds $\times$ (fabric $+ n\beta_{\text{in}}$) and a one-round algorithm wins by construction regardless of how many messages it holds in flight. The module's own default is `pairwise` and its docstring warns against `linear`; the selector recommends the opposite. The message counts in this document are unaffected — they come from `FORMULAS` rather than from `predict` — but the recommendation layer above them is not usable for this op until the time expression charges something per message.

Stepping outside the op family: the reason `alltoall` matters more than any other collective here is that its cost is the cost of a group chat, and 992 messages at $p = 32$ is what a fully connected conversation over thirty-two agents actually charges. Gunther's Universal Scalability Law has a

coherency coefficient κ multiplying $p(p-1)$, and an all-to-all exchange makes $\kappa > 0$ by construction, so throughput peaks and then declines rather than saturating. The alternative is not a better alltoall but a smaller graph. In the real eight-rank software run, `real-sw-p8-full`, the cross-review stage used `neighbor_allgather` and `neighbor_alltoall` over a degree-2 review graph and the fabric logged 16 messages for each — $2p$, against the $p(p-1) = 56$ this family would have charged at the same population. That is the contrast the project's thesis rests on: 56 against 16 at $p = 8$, $\Theta(p)$ against $\Theta(p^2)$ as p grows, and a bound on how much of every agent's context is other agents' output.

7 Threats to this reading

These runs contain no agents. `metrics.json` records no executor and the viewer's "agent calls" panel reads 0; the wall times of 0.014–0.959 s are SQLite writes and event-log appends. Every latency number above — the 250.6 rank-seconds, the $p^{3.71}$ fit, the 22.8 s straggler — is a property of the fabric under thread contention, not of model behaviour. In the agent regime α is seconds rather than microseconds, and the per-message contention that produces the super-quadratic fit would be swamped by invocation latency. The message counts are the durable result; the timings are indicative of shape only.

The imbalance may be a start-up artefact rather than a scheduling property. Ranks 10 and 16 at $p = 32$ being late is consistent with thread launch order, and I cannot separate "`linear` concentrates delay on the late rank" from "this particular run happened to start two ranks late". What supports the former is that it is a structural consequence of the schedule — a rank that posts all its sends before any receive cannot be slowed by a peer, only by its own lateness — and that `pairwise` at the same p with the same traffic shows a 2.4× spread rather than 29×. What would settle it is repeated runs at $p = 32$, and this sweep has one per campaign at that size.

The eager budget was lifted, so the transport hazard is untested. The peak queue of 31 unexpected messages per rank is a real measurement, but its consequences were suppressed by `eager_limit=10**9` and `strict_context=False`. A reader should not conclude from twenty-one clean runs that `linear` is safe at $p = 32$; the correct conclusion is that the message pattern is as predicted when flow control is disabled.

Round counts here are a convention, not an observation. Both the closed form and the implementation assert one round, and no measurement in the sweep could contradict them, because nothing counts rounds independently. The blocking analysis above is the closest thing to a check, and it suggests the single round is not free. The same convention is applied inconsistently elsewhere in the module — `bcast/flat` records $p - 1$ rounds at its root for a structurally identical fan-out — so the `rounds` column of the table should be read as "what the algorithm claims" rather than "what happened".